



CSI247: Data Structures

Lecture 02 – Review: Java Programming (The Basics)

Department of Computer Science
B. Gopolang (247-275)

Previous Lesson ...

- Course objectives, overview
- Topics to cover in the course
- Course material
- Course admin: schedule, pre-requisite, labs
- Assessments
- Useful tools
- Introduction to Data Structures

Today ...

Learning Objectives

In this lecture we will review:

- 1st year programming basics
 - Variables
 - Manipulating text
 - Control statements (selection and repetition)
 - Methods
 - Java defined Classes and Objects
 - API Documentation
 - Keyboard input
 - File IO

1. Variables

- Rem: Programs generally manipulate some data
 - Placeholders for data to be manipulated in memory
 - Must be declared before use (in Java)
 - Need to specify **data type**
 - A set of values with a set of permitted operations)
 - a) Primitives
 - e.g. **int**: +, *, ++, etc
 - b) Reference types (Classes)
 - e.g. **String**: charAt(), length(), indexOf(), etc
 - Memory diagram: Primitive vs Reference?

- Variable **declaration**
 - Allocate space for some data type and give it a name
 - E.g. *int x;*
- Variable **initialization**
 - Gives a variable an initial value
 - E.g. *x = 5;*
 - Use keyword **final** if it should not change
 - E.g.: *final int x = 5;*

2. Operators

Operator	Examples
Assignment	$x = 17;$ $x = x + 17;$ etc
Arithmetic	$+$, $-$, $*$, $/$, etc
Relational	$>$, $>=$, $<$, etc
Logical	$\&\&$, $!$, etc
Increment/decrement	$(++x, x--, \text{etc})$

- Note:
 - Order of precedence **Important!**

3. Strings

- Manipulating text
 - String class
 - Many possible operations via methods

e.g.

```
String dept = "Computer Science";
```

```
System.out.println(dept.toUpperCase());
```

```
System.out.println(dept.length() * 10);
```

```
int x = dept.indexOf("scie");
```

```
char ch = dept.charAt( dept.length() );
```

```
String st = dept.substring(9) + 7;
```

```
...
```


4. Control structures

a) Selection:

- if: *if* ($4 > 7$)
 $x += 4;$
- if – else : *if* ($4 > 7$)
 $x += 4;$
else
 $x += 8;$
- Multi-branch if
- Switch statement
- Nested if statements

b) Repetition:

- *for* & *while*
 - **Condition** statement
 - Body of the loop
 - **Update** statement
- Nested loops
- Various possible loop errors
 - E.g.: Infinite loop, one too many/less iterations, missing update, etc

5. Methods

- A named sequence of instructions.
- Has:
 - a) Name
 - b) Parameter list (and a data type for each)
 - Optional: 0 or several parameters
 - c) Return type
 - void
 - primitive or class type

d) Access specifier

- Indicates a method's visibility from other methods/classes



- Everywhere (default)



- Package members only



- Class members only

More
restrictive

- Method definition
 - For specifying method behaviour (logic)
 - A block of related statements:
 - Placed between this bracket pair: {...}

Access Specifier *Return type* *Method name* *Parameter list*

```
public int sum (int a, int c) {  
    return a+c;  
}
```

Method body

- Method invocation

int total = *sum* (4, 5);

- Method signature: *sum* (*int*, *int*)
- Used to differentiate between overloaded methods
- Method overloading
 - Multiple methods in a class with same name *but...*?
 - E.g. from String class

String word = "painkiller!";

System.out.println(word.substring(4, 7));

System.out.println(word.substring(4));

6. Classes and Objects

- Java: an OOP language
 - Related methods grouped into a unit called a **class**
 - A blueprint used to create **objects**
 - Rem:

Scanner scan = new Scanner(System.in); ??

- 2 major class categories in Java
 - Pre-defined classes (next slide)
 - User-defined classes (next handout)

7. Pre-defined classes

- Bundled into Java
 - A comprehensive list of pre-defined classes
 - Documentation of Java Library classes is accessible via:
<https://docs.oracle.com/javase/8/docs/api/>
 - It lists all Java classes (and packages)
 - For each class:
 - List of methods & their descriptions
- See next slide

Java™ Platform, Standard Edition 8 API Specification

This document is the API specification for the Java™ Platform, Standard Edition.

See: Description

Profiles

- compact1
- compact2
- compact3

Packages

Package

java.applet

Description

Provides the classes necessary to create an applet and the classes an applet uses to communicate with its applet context.

java.awt

Contains all of the classes for creating user

8. Keyboard input

- Scanner class
- E.g.

```
import java.util.Scanner;
```

...

```
Scanner scan = new Scanner(System.in);
```

```
System.out.println("Enter number of pens needed: ");
```

```
int pens = scan.nextInt();
```

```
System.out.println("Enter unit price: ");
```

```
double price = scan.nextDouble();
```

...

9. Displaying output

- Various options:
 - `System.out.print();`
 - Leaves cursor at end of displayed line
 - `System.out.println();`
 - Move cursor to beginning of next line
 - `System.out.printf()`
 - Uses formatting specifiers to format output
 - `%s`, `%5.2f`, `%c`, etc

10. File I/O

a) Reading from a File

- Several options available
- E.g.: FileReader (or File) class & Scanner class

```
import java.util.Scanner;
```

```
import java.io.*;
```

...

```
FileReader fReader = new FileReader("data.txt");
```

```
Scanner scan = new Scanner(fReader);
```

...

b) Writing to a File

- PrintWriter class

- E.g.

```
import java.io.*;
```

```
...
```

```
PrintWriter pw = new PrintWriter("out.txt");
```

```
pw.printf("Practicing File output");
```

```
pw.close();
```

Note

- i. Input file **MUST** exist
 - If not: **FileNotFoundException**
- ii. Output file may not exist
 - If not there: Created
 - If there: Overwritten by default!
- iii. You **MUST** close files when done
 - Why?

c) Catching file IO errors

- File operations are at risk of various file-related issues
 - E.g. file not existing
- Hence, catching File IO exceptions is vital
- Java has some File IO exception handling mechanisms
 - try-catch block
 - throws statement

Summary

- Review of some Java Programming basics
 - Methods, Using Java-defined classes & Java API documentation
- Next class
 - User Defined Classes
 - Arrays

Q & A



Thank you